NebulaScan: Automated Malware Detection Tool

Software Project Lab 3 Technical Report

Submitted by
Md. Rakibul Islam
BSSE 1210
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Mohammed Shafiul Alam Khan
Professor
Institute of Information Technology
University of Dhaka

Submission Date: 01 December 2024



Letter of Transmittal

01 December 2024

BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical report on “NebulaScan: Automated Malware Detection Tool”.

Sir,
With due respect, I am pleased to submit the technical report of Software Project Lab 3 on “NebulaScan: Automated Malware Detection Too”. I have tried my best to deliver a good understanding about the technical details of the project. However, it might lack perfection. So, I, therefore, hope that you would be kind enough to accept my report and oblige thereby.

Sincerely yours,
Md. Rakibul Islam
Roll: BSSE 1210
Institute of Information Technology
University of Dhaka

_____

Approved By
Dr.Mohammed Shafiul Alam Khan
Professor
Institute of Information Technology
University of Dhaka

Acknowledgment

Firstly, I would like to thank the Almighty for helping me complete the final report. I am grateful to the Institute of Information Technology for giving such a tremendous opportunity to work on “NebulaScan: Automated Malware Detection Too”. I would like to convey my tremendous gratitude to my supervisor, Dr.Mohammed Shafiul Alam Khan,Professor, Institute of Information Technology, University of Dhaka, for providing me with guidelines about preparing this report. She helped me a lot by sharing his valuable knowledge with me and monitored my task throughout the whole time. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

Nebula is a Transformer-based neural architecture designed to detect Windows malware by analyzing dynamic execution logs. Unlike prior models focused on homogeneous features like runtime APIs, Nebula integrates diverse data types like kernel syscall, module entry type, API calls, API arguments, and return value for more comprehensive analysis. It includes components for tokenization, filtering, normalization, and encoding to preprocess input for the Transformer. The system performs ablation studies to optimize these components and utilizes executables's hash to avoid redundant simulations, saving time. Additionally, a GUI enables users to input samples, and generate detailed malware analysis reports.

Table of Contents

Letter of Transmittal..... 1
Acknowledgment..... 2
Abstract..... 3
Table of Contents..... 4
1. Introduction..... 1
1.1 Motivation..... 1
1.2 Scope..... 2
1.3 Project Timeline..... 2
1.4 Purpose of Document..... 2
1. Project Description..... 3
2.1. Quality Function Deployment..... 3
2.1.1 Normal Requirements..... 3
2.1.2 Expected Requirements..... 3
2.1.3 Exciting Requirements..... 3
2.2 Usage Scenario..... 4
3. Scenario based Modeling..... 5
3.1 Use Case Diagram..... 5
3.1.1 NebulaScan..... 5
3.1.2 Modules..... 6
3.1.2.1 Upload Executable..... 6
3.1.2.2 Dynamic Log Extraction..... 7
3.1.2.3 Preprocessing..... 7
3.1.2.4 Malware Detection..... 7
3.1.2.5 Generate Reports..... 7
4. Data Based Modeling..... 8
5. Class Based Modeling..... 9
5.1 Analysis Classes..... 9
5.2 Class Cards..... 10
5.3 Class Responsibility and Collaboration Diagram..... 13
6. Architectural Design..... 14
6.1 Architectural Context Design..... 14
6.2 Define Archetypes..... 15
6.3 Top level component..... 16
6.4 Instantiation of the Components..... 16
7. Preliminary Test Plan..... 17
7.1 High-Level Description of Testing Goals..... 17
7.2 Test Cases..... 17
Test Case 1: System Initialization..... 17
Test Case 2: Dynamic Log Extraction..... 18
Test Case 3: Normalization Testing..... 18
Test Case 4: Tokenization Testing..... 19

Test Case 5: Classification..... 19
Test Case 6: Report Generation..... 20
Test Case 7: System Performance..... 20
Test Case 8: Fault Tolerance..... 21
Test Case 9: User Experience..... 21
8. Conclusion..... 22
References..... 23

List of Figures

Figure-01: Timeline for NebulaScan 2
Figure-02: UseCase Level 0 (NebulaScan) 5
Figure-03: UseCase Level 1 (Modules of NebulaScan) 6
Figure-04: UseCase Level 1.3(Preprocessing) 7
Figure-05: CRC Diagram of NebulaScan 13
Figure-06: Architectural Context Diagram 14
Figure 07: Pipe & Filter Pattern 15
Figure-08: Top Level Components 16
Figure-09: Detect Maliciousness component(Pipe & Filter Pattern) 16

List of Tables

Table-1: Schemas of NebulaScan 8
Table-2: CRC Card (Executable) 10
Table-3: CRC Card (DynamicLogExtractor) 10
Table-4: CRC Card (Normalization) 11
Table-5: CRC Card (Tokenization) 11
Table-6: CRC Card (NebulaTransformerModel) 12
Table-7: CRC Card (ReportGenerator) 12

1. Introduction

DYNAMIC malware analysis is a crucial task not only for detecting but also for understanding the threats that are widespread over the entire Internet. Once samples are collected, analysts execute malware inside isolated environments (sandboxes or emulators), where they list all the actions performed by the program like **Module entry**, **API calls**, **API arguments**, **API Return value**, and **kernel syscalls.** These actions are then summarized into textual reports, which are manually analyzed by experts to distill the rationale behind the maliciousness of the analyzed sample[1].

While models like Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks have been used to extract patterns from textual data, these methods face critical challenges. CNNs are limited to capturing local relationships within the data, and LSTMs struggle with lengthy sequences, such as extended chains of API calls [2]. To overcome these limitations, we implement **NebulaScan**, a comprehensive malware analysis solution leveraging Transformer architecture for enhanced detection and classification. NebulaScan integrates dynamic log generation via the **Speakeasy emulator[3]**, preprocessing pipelines for cleaning and tokenization, and a self-attention-based Transformer model. Unlike traditional approaches, NebulaScan handles heterogeneous data, effectively analyzing complex relationships between malware behaviors [2].

Complementing its technical capabilities, NebulaScan features a user-friendly desktop interface, automated hash caching for efficient emulation, and robust report generation to provide actionable insights. By streamlining the analysis process and enhancing accuracy, NebulaScan not only advances malware detection but also contributes to the broader field of cybersecurity.

1.1 Motivation

Dynamic malware analysis is critical in cybersecurity but often struggles with detecting sophisticated threats due to limitations in traditional methods, such as missing global patterns in dynamic logs. **NebulaScan** addresses these issues by leveraging Transformer-based machine learning models that effectively capture both local and global relationships within complex datasets.

By integrating dynamic log generation, preprocessing pipelines, and a cutting-edge detection system, NebulaScan significantly enhances the accuracy and efficiency of malware classification. This project not only improves detection capabilities but also demonstrates the powerful synergy between machine learning and malware analysis, providing a robust solution against evolving cyber threats.

1.2 Scope

The scope of **NebulaScan** is below

- A User-Friendly Desktop Application
- Dynamic Log Generation using an emulator(Speakeasy)
- Advanced Data Preprocessing(BPE tokenization)
- Transformer-Based Machine Learning Model
- Hash Caching Mechanism to avoid redundant emulation
- Comprehensive Reporting Module

1.3 Project Timeline

The estimated timeline of NebulaScan is given below:

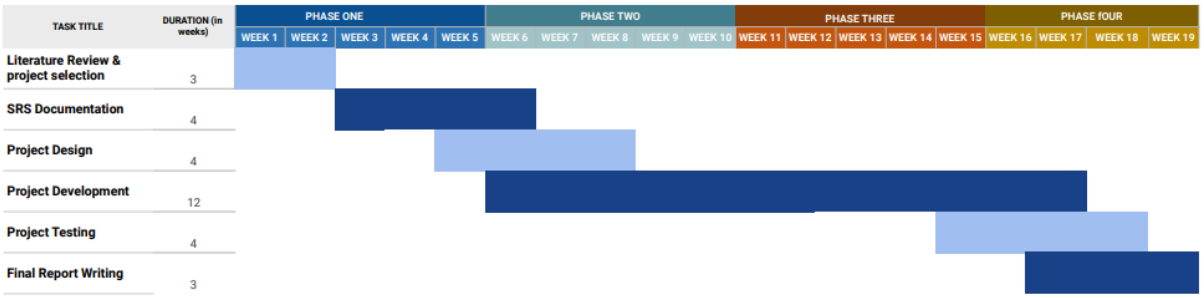


Fig 01: Project Timeline

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Reduce the development effort
- Improve understanding

1. Project Description

NebulaScan’s Quality Function Deployment (QFD) was shaped by an in-depth review of research, including insights from Max and Sejfia's work[4], and active engagement with stakeholders in software development and security. Feedback from developers across various software firms helped align the project with industry needs and concerns.

2.1. Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified-

2.1.1 Normal Requirements

1. **Automated Malware Detection**: The application should seamlessly detect malware in executable files using a Transformer-based model without requiring manual intervention.
2. **Data Preprocessing**: Includes robust pipelines for cleaning and tokenizing logs to ensure accurate analysis and model predictions.
3. **Dynamic Log Generation**: Automatically generates logs during analysis using the Speakeasy emulator, ensuring comprehensive data collection.
4. **Basic Reporting**: Generates basic reports summarizing detection outcomes, including whether the file is malicious or benign.

2.1.2 Expected Requirements

1. **User-Friendly Desktop Interface**: Provides an intuitive GUI for users to input executables and view results.
2. **Detection Accuracy**: The Transformer model should exhibit high accuracy in classifying executables as malicious or benign with minimal false positives or negatives.

2.1.3 Exciting Requirements

1. **Hash Caching**: Implements caching to skip redundant analysis of previously processed files, improving overall efficiency.

2.2 Usage Scenario

“NebulaScan: Dynamic Malware Detection System” is a user-friendly desktop application designed to analyze executable files for potential malware threats. The system integrates dynamic malware analysis, preprocessing, and Transformer-based classification to provide accurate and actionable insights to users. The usage scenario for NebulaScan is outlined as follows:

File Upload and Analysis Initiation
Users can launch the NebulaScan application and upload executable files directly through the intuitive graphical interface. Once uploaded, the system initiates dynamic log generation using the open-source emulator Speakeasy, capturing runtime behaviors of the file.

Automated Detection
NebulaScan automatically processes the generated logs through cleaning and tokenization pipelines before analyzing them with its Transformer-based malware detection model. The model classifies the file as benign or malicious and provides immediate feedback within the interface. Suspicious files are flagged with detailed threat classifications.

Hash Caching and Efficiency
To improve efficiency, NebulaScan employs a hash-caching mechanism that remembers previously analyzed files. If a file has been processed before, the system retrieves the cached results without re-emulating, saving significant time for users handling large datasets.

Comprehensive Report Generation
Upon completing the analysis, NebulaScan generates a detailed report summarizing the findings. This includes classification results, threat indicators, and statistical insights into the file's behavior. The reports can be exported for documentation or further review.

3. Scenario based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 NebulaScan

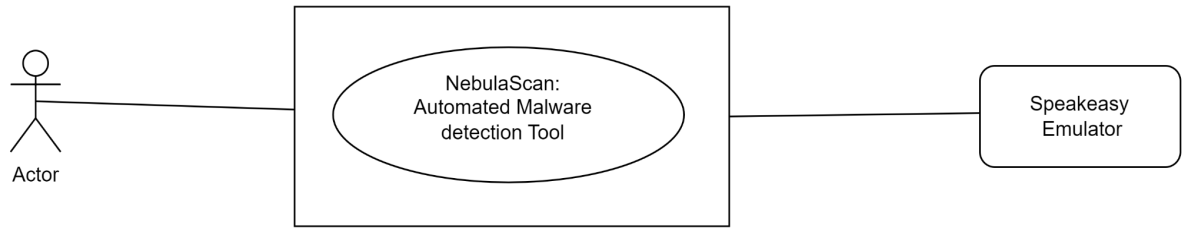


Fig 02: Use Case level 0(NebulaScan)

Primary actors: User
Secondary actors: Speakeasy Emulator
Goal in Context: The diagram shown above represents the whole Automated and Practical “NebulaScan Automated Malware Detection System”.

3.1.2 Modules

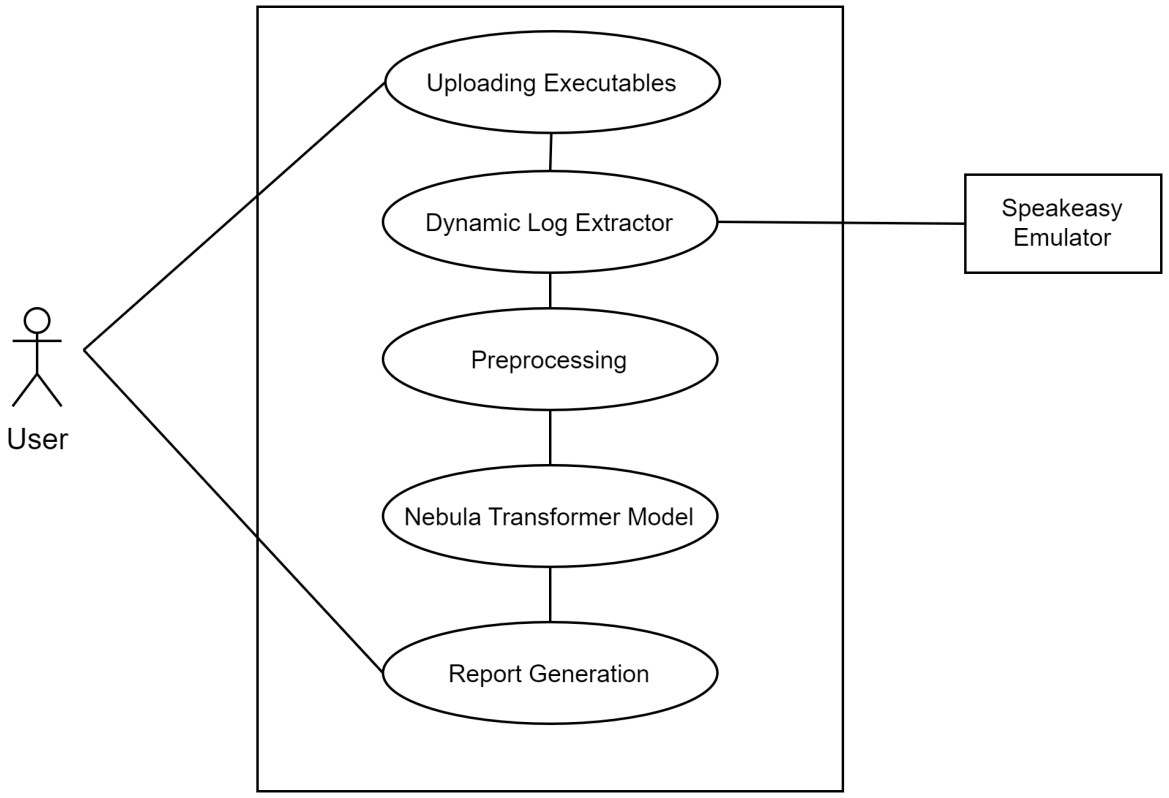


Fig 03: Use case level 1(modules of NebulaScan)

Primary Actor: User
Secondary Actor: Speakeasy Emulator
Goal in Context: The diagram shows the modules of NebulaScan. NebulaScan consists of 5 modules.
The modules can be elaborated as follows

3.1.2.1 Upload Executable

This module allows users to upload Windows executable files(.exe) for analysis. It ensures secure file handling and prepares the data for feature extraction and malware detection.

3.1.2.2 Dynamic Log Extraction

The "Dynamic Log Extraction" module is responsible for collecting dynamic execution logs by running executable files in a controlled environment(Speakeasy Emulator). It captures detailed behavior data of the software, including system calls, module entry type, api name, api arguments, api returns address etc.

3.1.2.3 Preprocessing

The "Preprocessing" module processes raw execution logs, extracting meaningful features for analysis. Techniques include tokenization, data normalization, preparing the data for input into the machine learning model.

This module can be divided into two submodules Normalization & tokenization

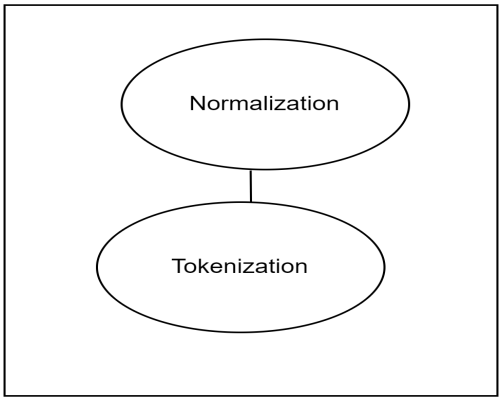


Fig 04: Use Case level 1.3 (Preprocessing)

3.1.2.4 Malware Detection

The "Malware Detection" module serves as the core of NebulaScan's functionality. It leverages Transformer-based machine learning models to analyze dynamic logs and detect malicious patterns with high accuracy. If suspicious behavior is identified, the module triggers further investigation.

3.1.2.5 Generate Reports

The "Generate Reports" module creates detailed analysis reports for detected malware. These reports explain why the sample was flagged as malicious, including specific patterns or anomalies observed during analysis, enabling users to understand the risks better.

4. Data Based Modeling

Database modeling represents the structure of data in a visual format, organizing it into entities, attributes, and relationships. The process starts by identifying key entities based on system requirements and mapping out their relationships using tools like Entity-Relationship Diagrams (ERDs). These diagrams guide the database design, ensuring efficient data storage and retrieval.

For **NebulaScan**, the primary data objects derived from its functionality and usage scenarios are:

1. Executable

Schema for the executable is shown below:

Schema	**Attributes**	**Type**	**Description**
Executable	Executable_ID(PK)	INT (Auto Increment)	Unique identifier for the executable.
	Executable_Name	VARCHAR(255)	Name of the executable file.
	SHA256_Hash	VARCHAR(255)	SHA-256 hash of the executable, used for unique identification.
	File_path	VARCHAR(255)	Path to the executable file in the system.
	File_Size	VARCHAR(255)	Size of the executable file bytes.
	Entry_Points	VARCHAR(255)	Entry points or starting points of the executable.
	Api	Object	List of the api names
	Error_Logs	object	Logs of any errors or exceptions encountered during analysis.

	Maliciousness_Probability	Decimal(6)	score indicating the probability of maliciousness
	Classification_Label	ENUM('benign', 'malicious')	Classification label indicating if the executable is benign or malicious.
	Analysis_Date	TIMESTAMP	Date and time when the executable was analyzed.

Table 01: Schema of Executable Entity

5. Class Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. Executable
2. DynamicLogExtractor
3. Normalization
4. Tokenization
5. NebulaTransformerModel
6. ReportGenerator

5.2 Class Cards

The class cards of NebulaScan is given below:

Executable	
Attributes	**Methods**
-executable_id -executable_name -executable_path -exceutable_hash -file_path -file_size -entry_points -apis -erros_logs -maliciousness_probability -classification_label	+getter() +setter() +extract_file_info() +generate_api_hash() +check_pre_executed()
Responsibilities	**Collaborators**
● Represents the executable file. ● Extracts metadata and relevant information from the executable for analysis.	DynamicLogExtractor, Normalization, Tokenization, NebulaTransformerModel

Table 02: CRC Card(Executable)

DynamicLogExtractor	
Attributes	**Methods**
-executable -speakeasy_config -log_file -output_folder	+file_path_check() +is_executable_file() +emulate() +save_emulation_file() +create_error()
Responsibilities	**Collaborators**
● Extracts dynamic execution logs from the executable.	Executable

Table 03: CRC Card(Dynamic Log Extractor)

Normalization	
Attributes	**Methods**
-executable -non_normalized_data -normalized_data	+filter() +normalize() +normalizeIP() +normalizeStringHash() +normalizeStringPath()
Responsibilities	**Collaborators**
● Preprocesses raw data into a format suitable for model input. ● Ensures that the data is scaled and normalized to improve model accuracy.	Executable, DynamixLogExtractor

Table 04: CRC Card(Normalization)

Tokenization	
Attributes	**Methods**
-executable -sequence_len -vocabulary_size -special_token -pad_token -unk_token -mask_token -stop_symbols	+clear_special_characters() +pad_sequence() +tokenize() +build_vocabulary() +load_tokenizer() +split_string_chunks() +encode()
Responsibilities	**Collaborators**
● Tokenizes raw binary data to prepare it for model analysis. ● Ensures tokens are consistent and properly formatted.	Executable, Normalization, NebulaTransformer

Table 05: CRC Card(Tokenization)

NebulaTransformerModel	
Attributes	**Methods**
-executable -model -optimizer -batch_size -loss_function -tokenizer -model_path	+train_model() +predict() +load_pretrainned_model()
Responsibilities	**Collaborators**
● Performs binary classification (benign or malicious) using the transformer model. ● Trains the model on preprocessed data and stores predictions. ● Predicts probability of any executables	Executable, Normalization, Tokenization

Table 06: CRC Card(NebulaTransformerModel)

ReportGenerator	
Attributes	**Methods**
-executable	+generateReport()
Responsibilities	**Collaborators**
● Creates detailed reports for user review.	Executables

Table 07: CRC Card(ReportGenerator)

5.3 Class Responsibility and Collaboration Diagram

The CRC diagrams of the classes is given below:

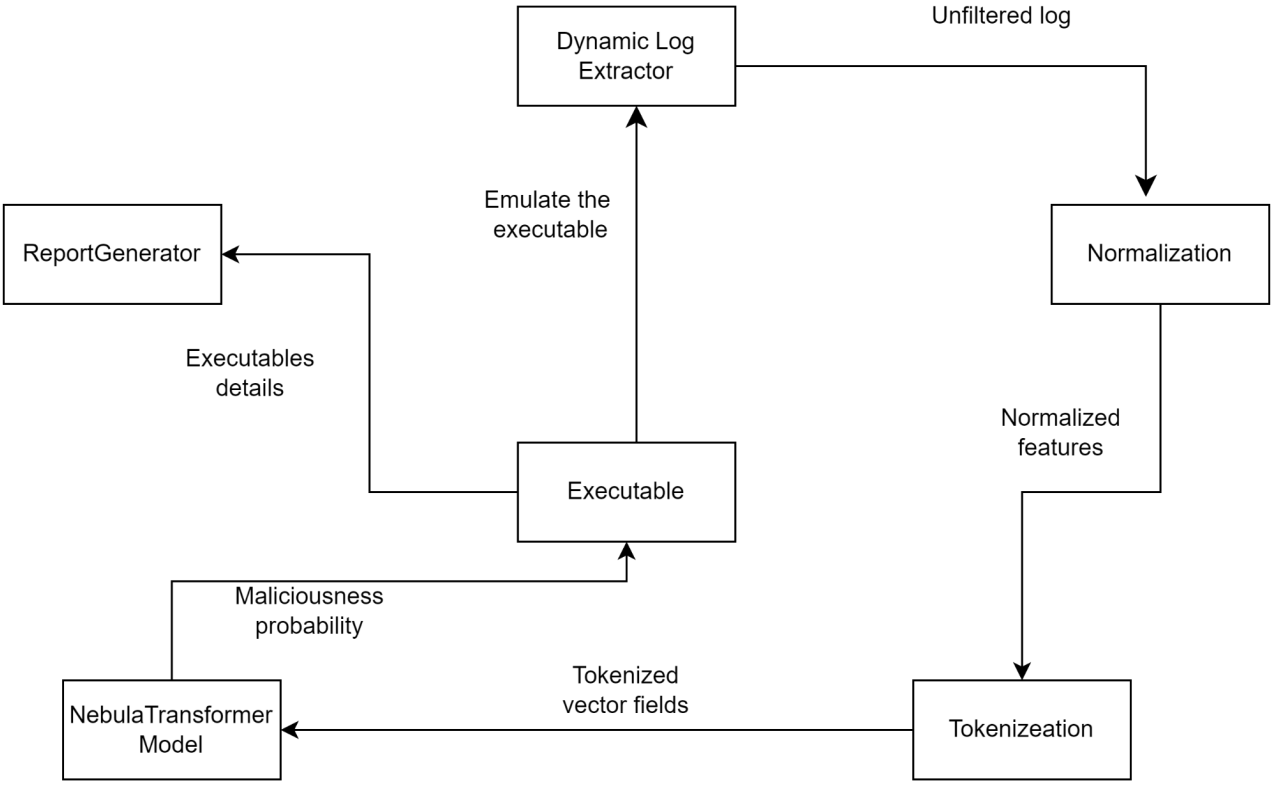


Fig 05: CRC Diagrams of NebulaScan

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Design

The architectural context diagram for the tool is given below-

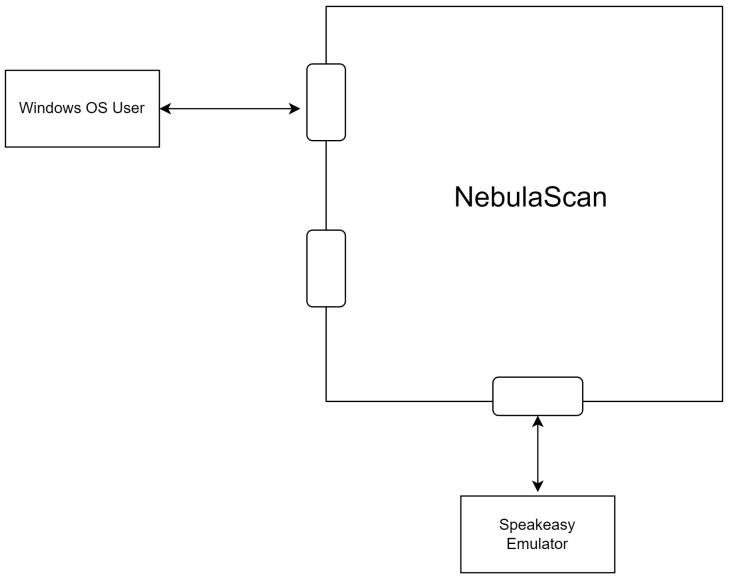


Fig 06: Architectural Context Design

Two categories of users are supposed to be the target for the tool. They are Windows operating system users and researchers. Both types of users are the producers and also consumers of the information through the tool. There is one subordinate system, Speakeasy Emulator. The system will use the speakeasy emulator to extract dynamic log from executables.

6.2 Define Archetypes

An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, relatively small set archetypes are required to design even relatively complex systems.In many cases, archetypes can be derived by examining the analysis classes defined as part of the requirements model. In our cases, **Pipe & Filter Architecture** is used.

Pipe and Filter Architecture

The Pipe and Filter architecture is a design pattern that divides a process into a series of distinct steps, called filters, linked by channels known as pipes. Each filter is dedicated to a particular processing function, whether it involves transforming, validating, or aggregating data. Data flows through these filters via pipes, which carry the output of one filter to the input of another.

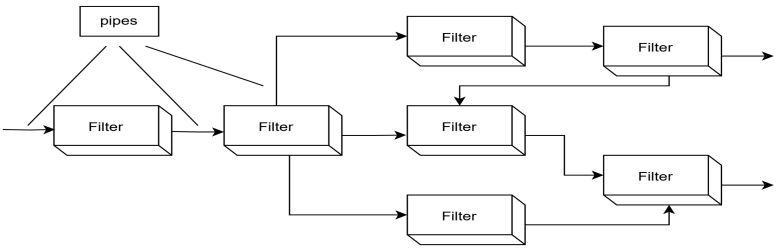


FIg 07: Pipe & Filter Architecture

- **Filters**: Each filter carries out a specific, standalone task, whether it be transforming, validating, or processing data before forwarding it. In the setup, there are two levels of filters; the first one processes data from the pump and hands it to the second filter, which continues the processing before passing it on.
- **Pipes**: Pipes serve as the channels for data movement between filters, linking each component in sequence and ensuring a smooth transfer of data. In diagrams, pipes are depicted as arrows connecting the components.

6.3 Top level component

Overall architectural structure with top level components is illustrated below-

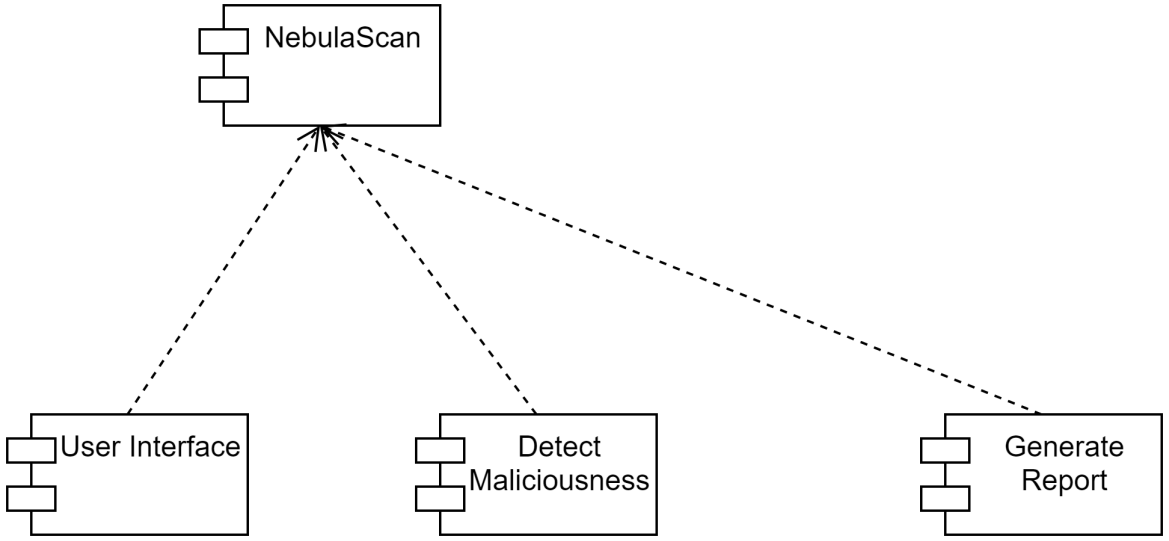


Fig 08: Top Level components

6.4 Instantiation of the Components

We apply architecture on the components to make a maintainable collection of components. In our case, Detect maliciou
Pattern: Pipe & Filter Pattern

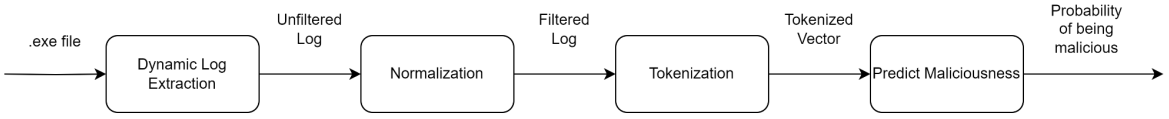


Fig 09: Detect Maliciousness component(Pipe & Filter Pattern)

The **Pipe and Filter pattern** is fitting perfectly to ensure a clean and modular data processing workflow, as illustrated in the diagram. Each block in the pipeline represents a **filter** that performs a distinct operation, and the arrows between them indicate the **pipes** that transfer data sequentially. Here's how it fits:

1. **Dynamic Log Extraction**: Acts as the first filter, extracting raw behavioral data from the logs. This raw data serves as input for subsequent filters.
2. **Normalization**: The next filter processes the extracted data to standardize it, ensuring uniformity for accurate tokenization.
3. **Tokenization**: Converts normalized data into a tokenized format suitable for model input, adding another layer of transformation.
4. **Predict Maliciousness**: The final filter leverages the processed data to predict and classify maliciousness using the transformer-based model.

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High-Level Description of Testing Goals

The testing goals for this tool are:

- To demonstrate that the system meets its requirements.
- To verify that the dynamic logs are extracted accurately from executable files.
- To validate the normalization functionality correctly.
- To validate the tokenization functionality correctly
- To ensure the Transformer-based architecture classifies executables accurately.
- To test the detailed report generation for predictions.
- To ensure the overall system operates smoothly with minimal false positives and negatives.
- To validate system performance with varying workloads.

7.2 Test Cases

Test ID: T1

Test Case 1: System Initialization

- **Test Scenario**: Start the system with no prior configurations.
- **Steps**:
    1. Load the application for the first time.
    2. Verify if all necessary components initialize without error.
- **Expected Outcome**: The application initializes successfully and is ready for analysis.
- **Target Item**: Application startup process.

Test ID: T2

Test Case 2: Dynamic Log Extraction

- **Test Scenario**: Analyze an executable file to extract dynamic logs.
- **Steps**:
    1. Provide an executable file to the system.
    2. Run the `DynamicLogExtractor`.
    3. Verify the extracted log data.
- **Expected Outcome**: Dynamic logs are extracted without data loss or errors.
- **Target Item**: Dynamic log extraction.

Test ID: T3

Test Case 3: Normalization Testing

Test Scenario: Validate that the normalization process correctly standardizes dynamic log data.
Steps:

1. Provide dynamic log data with varied formatting, inconsistent values, and irregularities.
2. Run the normalization step on the provided data.
3. Verify that the data is standardized (e.g., consistent formats, removal of noise, or scaling applied).
   Expected Outcome:
   The data is cleaned and standardized, with consistent formatting and values suitable for further processing.
   Target Item: Normalization process in the preprocessing pipeline

Test ID: T4

Test Case 4: Tokenization Testing

Test Scenario: Validate that the tokenization process correctly splits dynamic log data into meaningful tokens.
Steps:

1. Provide normalized dynamic log data.
2. Run the tokenization step on the data.
3. Verify that the data is correctly tokenized into individual tokens based on predefined rules (e.g., splitting by delimiters or extracting features).
   Expected Outcome:
   The data is split into meaningful and accurate tokens, ready for input into the machine learning model.
   Target Item: Tokenization process in the preprocessing pipeline.

Test ID: T5

Test Case 5: Classification

- **Test Scenario**: Classify executables as benign or malicious.
- **Steps**:
  1. Provide preprocessed data to the transformer model.
  2. Run the `NebulaTransformer` for prediction.
  3. Verify classification accuracy.
- **Expected Outcome**: The model predicts correctly with minimal false positives and negatives.
- **Target Item**: Transformer-based classification.

Test ID: T6

Test Case 6: Report Generation

- **Test Scenario**: Generate a detailed report for a classified executable.
- **Steps**:
    1. Run classification on an executable.
    2. Use `ReportGenerator` to create a report.
    3. Verify the report's accuracy, detail, and clarity.
- **Expected Outcome**: The report provides accurate and comprehensive information.
- **Target Item**: Report generation.

Test ID: T7

Test Case 7: System Performance

- **Test Scenario**: Analyze performance with varying workloads.
- **Steps**:
    1. Provide multiple executables of varying sizes and complexities.
    2. Observe and measure system responsiveness and analysis time.
- **Expected Outcome**: The system performs efficiently with minimal latency.
- **Target Item**: Overall system performance.

Test ID: T8

Test Case 8: Fault Tolerance

- **Test Scenario**: Verify the system's behavior with corrupted or unsupported files.
- **Steps**:
    1. Provide a corrupted or unsupported executable file.
    2. Run the system analysis.
- **Expected Outcome**: The system handles errors gracefully and logs them without crashing.
- **Target Item**: Error handling and fault tolerance.

Test ID: T9

Test Case 9: User Experience

- **Test Scenario**: Validate the user interface and workflow.
- **Steps**:
    1. Navigate through the application.
    2. Perform an end-to-end analysis with feedback submission.
- **Expected Outcome**: The workflow is intuitive, and the interface is user-friendly.
- **Target Item**: User interface and experience.

8. Conclusion

In this report, I have discussed all the necessary technical details to develop "NebulaScan: Automated Malware Detection Tool." I have elaborated on the scope, assumptions, usage scenarios, and system requirements. Scenario-based modeling, data modeling, and class-based modeling have been performed to design and define the components of the tool. The architectural design and detailed implementation methodology have been outlined, ensuring a clear depiction of the workflow. Furthermore, I have presented a high-level description of testing goals along with test cases to validate the tool's functionality.

While NebulaScan aims to address critical challenges in dynamic malware analysis with advanced transformer-based architecture, there is potential for future improvements. Expanding feature sets, optimizing performance, and incorporating support for additional file types can enhance its capabilities.

This document provides a comprehensive overview of the system and serves as a guide to understanding the technical foundation of NebulaScan.

References

[1] M. Cinque, D. Cotroneo, and A. Pecchia, “Challenges and directions in security information and event management (SIEM),” in Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops (ISSREW), Oct. 2018, pp. 95–99.

[2] Trizna, Dmitrijs, Luca Demetrio, Battista Biggio, and Fabio Roli. "Nebula: Self-Attention for Dynamic Malware Analysis." *IEEE Transactions on Information Forensics and Security* (2024).

[3] Speakeasy Emulator: *https://github.com/mandiant/speakeasy*

[4] Adriana Sejfia, Max Schäfer. 2022. Practical Automated Detection of Malicious npm Packages. In 2022 International Conference in Software Engineering (ICSE). 1681–1692. https://arxiv.org/pdf/2202.13953.pdf